

Velocity Corporate Training Center, Pune

Sequence Modeling in NLP – RNN, LSTM, and GRU



NLP Content

1. Introduction to NLP

- Definition & Importance
 - Common NLP Tasks and Applications
 - Structured vs. Unstructured Data
 - Challenges in NLP
-

2. Text Preprocessing

- Tokenization (Word, Sentence, Subword)
 - Normalization (Lowercasing, Removing Punctuation & Special Characters)
 - Stopword Removal
 - Lemmatization vs. Stemming
 - Text Cleaning (Regex, Emojis, HTML Tags)
-

3. Feature Extraction & Text Vectorization

- Traditional Encoding Techniques:
 - One-Hot Encoding
 - Bag of Words (BoW)
 - TF-IDF (Term Frequency–Inverse Document Frequency)
 - Integer Encoding
 - Embedding-Based Techniques:
 - Word2Vec (Skip-Gram, CBOW)
 - GloVe
 - FastText
 - Trainable Embeddings vs. Pretrained Embeddings
 - Limitations and Practical Comparisons
-

4. Sequence Modeling Basics

- Need for Sequential Modeling in NLP
 - Padding and Truncation Techniques
 - Embedding Layer Role and Usage
 - Input Types: Integer Encoding, One-hot, Dense Embeddings
-

5. Recurrent Neural Networks (RNNs)

- RNN Architecture & Working Principle
 - Hidden State & Forward Propagation
 - Backpropagation Through Time (BPTT)
 - Challenges: Vanishing & Exploding Gradients
 - RNN Input-Output Types:
 - One-to-Many (Text Generation)
 - Many-to-One (Sentiment Analysis)
 - Many-to-Many (POS Tagging, NER)
 - Bidirectional RNN
 - Stacked RNN (Deep RNNs)
 - Practical RNN Implementation (Keras/PyTorch Examples)
-

6. Long Short-Term Memory (LSTM)

- LSTM Architecture (Forget, Input, Output Gates)
- Cell State vs. Hidden State
- Advantages Over Vanilla RNN
- Bidirectional LSTM
- Stacked LSTM
- Practical LSTM Implementation (Keras/PyTorch Examples)

7. Gated Recurrent Unit (GRU)

- GRU Architecture (Update & Reset Gates)
- Differences Between GRU and LSTM
- When to Use GRU Over LSTM
- Bidirectional GRU
- Stacked GRU
- Practical GRU Implementation (Keras/PyTorch Examples)

8. Comparison between RNN, LSTM and GRU

- Core Comparison – RNN vs. LSTM vs. GRU
- Advanced Comparison – Stacked, Bidirectional, and Variants

◆ 1. Introduction to NLP

📄 What is NLP?

Natural Language Processing (NLP) is a field of AI that deals with the interaction between computers and humans using natural language (text or speech). It enables machines to:

- Read, understand, interpret, and generate human language.

📌 Examples of NLP Tasks:

Task	Description	Example
Text Classification	Categorize text into predefined labels	Spam detection, topic classification
Sentiment Analysis	Detect emotion in text	"The movie was awesome!" → Positive
Named Entity Recognition (NER)	Identify entities in text	"Apple Inc. was founded by Steve Jobs" → Apple = Org, Steve = Person
Machine Translation	Translate between languages	English to French
Question Answering	Extract answers from context	Chatbots, Google Search
Text Summarization	Generate summary of large texts	News summarizer

◆ 2. Text Preprocessing

Raw text must be cleaned and standardized before feeding it into models.

2.1 Tokenization

Split text into smaller units (tokens).

◆ Word Tokenization

```
from nltk.tokenize import word_tokenize
text = "ChatGPT helps with NLP tasks!"
word_tokenize(text) # ['ChatGPT', 'helps', 'with', 'NLP', 'tasks', '!']
```

◆ Sentence Tokenization

```
from nltk.tokenize import sent_tokenize
text = "Hello. I am learning NLP."
sent_tokenize(text) # ['Hello.', 'I am learning NLP.']
```

2.2 Normalization

◆ Lowercasing

```
text.lower()
```

◆ Stemming

Removes suffixes:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
ps.stem("running") # run
```

◆ Lemmatization

Returns root form (using POS tagging):

```
from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()
lemmatizer.lemmatize("running", pos="v") # run
```

2.3 Stopword Removal

Remove common, uninformative words:

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
```

2.4 Text Cleaning

- Remove punctuations: `re.sub(r'^\w\s', '', text)`
- Remove extra spaces, emojis, HTML tags, etc.

◆ 3. Feature Extraction & Text Vectorization

Text vectorization is the process of converting text into numerical format so that it can be processed by machine learning and deep learning models. Since models work with numbers, vectorization acts as a critical bridge between human language and algorithms.

3.1 Traditional Text Encoding Techniques (Manual/Statistical Methods)

Why Do We Need Text Encoding?

Machine Learning and Deep Learning models cannot understand raw text. They only process numerical input. So, we must convert text into numbers while preserving as much useful information as possible.

This process is called **text encoding** or **feature extraction** in Natural Language Processing (NLP).

Example:

1. “We are learning Python and Data Science at Velocity”
2. “Python language is used for Machine Learning and Deep Learning”

To analyze these using machine learning (like sentiment analysis or clustering), we must first convert the text into numerical format using one of the encoding techniques below.

3.1.1 Integer Encoding

Concept

Each **unique word** is assigned a unique integer ID. No semantic meaning is encoded — it's simply a mapping.

For example, "python" could be 4 and "velocity" could be 9. This doesn't mean 9 is more important than 4.

Example Vocabulary Mapping

```
{'we': 1, 'are': 2, 'learning': 3, 'python': 4, 'and': 5,
 'data': 6, 'science': 7, 'at': 8, 'velocity': 9}
```

Code Example

```
from keras.preprocessing.text import Tokenizer

sentences = [
    "We are learning Python and Data Science at Velocity",
    "Python language is used for Machine Learning and Deep Learning"
]
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(sentences)
encoded = tokenizer.texts_to_sequences(sentences)

print("Word Index:", tokenizer.word_index)
print("Encoded Sentences:", encoded)
```

3.1.2 One-Hot Encoding (Word-Level)

Concept

Each word is represented as a binary vector with one "1" and the rest "0"s. The length of the vector is equal to the vocabulary size.

This approach results in **high-dimensional and sparse** vectors.

Example

If the vocabulary size is 5 and "python" is the second word:

```
[0, 1, 0, 0, 0]
```

Code Example

```
from keras.utils import to_categorical

word_index = {'python': 2, 'data': 3}
word_id = word_index['python']
one_hot_vector = to_categorical(word_id, num_classes=len(word_index) + 1)

print("One-hot vector for 'python':", one_hot_vector)
```

3.1.3 Bag of Words (BoW)

Concept

Bag of Words creates a fixed-size vocabulary from the text corpus and represents each sentence as a vector based on **word frequency**.

It does **not preserve word order or context**, but it gives us a basic way to convert text into numbers.

Simple Formula

Let:

- V = total number of unique words in the vocabulary
- d = a document (sentence)

Then, the **BoW vector** for the document d is:

$$\text{BoW}(d) = [\text{Count}(w_1), \text{Count}(w_2), \dots, \text{Count}(w_V)]$$

Where $\text{Count}(w_i)$ is the number of times word w_i appears in document d .

Vocabulary (Example)

```
['we', 'are', 'learning', 'python', 'and', 'data', 'science', 'at', 'velocity',
 'language', 'is', 'used', 'for', 'machine', 'deep']
```

Frequency Matrix

Sentence	we	are	learning	python	and	data	science	at	velocity	language	is	used	for	machine	deep
"We are learning Python and Data Science at Velocity"	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0
"Python language is used for Machine Learning and Deep Learning"	0	0	1	1	1	0	0	0	0	1	1	1	1	1	1

Code Example

```
from sklearn.feature_extraction.text import CountVectorizer

sentences = [
    "We are learning Python and Data Science at Velocity",
    "Python language is used for Machine Learning and Deep Learning"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(sentences)
```

```
print("Vocabulary:", vectorizer.get_feature_names_out())
print("BoW Matrix:\n", X.toarray())
```

3.1.4 TF-IDF (Term Frequency – Inverse Document Frequency)

Concept

TF-IDF improves BoW by down-weighting **common words** and giving more importance to **unique or rare words**.

It is widely used in text mining, information retrieval, and NLP tasks such as document classification and keyword extraction.

◆ TF (Term Frequency)

Represents how often a word appears in a document relative to the total number of words in that document.

$$TF(t, d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

◆ IDF (Inverse Document Frequency)

Represents how rare or unique a word is across all documents in the corpus.

$$IDF(t) = \log\left(\frac{N}{1 + DF(t)}\right)$$

Where:

- N = Total number of documents
- $DF(t)$ = Number of documents containing term t

Adding 1 in the denominator avoids division by zero when a term appears in all documents.

◆ Combined Formula

The TF-IDF score is computed as:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$$

Code Example

```
from sklearn.feature_extraction.text import TfidfVectorizer

sentences = [
    "We are learning Python and Data Science at Velocity",
    "Python language is used for Machine Learning and Deep Learning"
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(sentences)

print("Vocabulary:", vectorizer.get_feature_names_out())
print("TF-IDF Matrix:\n", X.toarray())
```

3.1.5 Comparison of Encoding Techniques

Technique	Preserves Order	Captures Frequency	Captures Importance	Sparse?	Semantic Meaning
Integer Encoding	No	No	No	No	No
One-Hot Encoding	No	No	No	Yes	No
Bag of Words	No	Yes	No	Yes	No
TF-IDF	No	Yes	Yes	Yes	No

3.1.6 Limitations of Traditional Encoding Techniques

- Cannot capture **word meaning** (e.g., "good" vs. "excellent")
- Do not understand **context** (e.g., "bank" as river vs finance)
- Ignore **word order**
- Generate **sparse and high-dimensional** vectors
- Cannot handle **unseen (OOV)** words in test data

3.2 Word Embeddings (Dense Vector Representations)

Why Word Embeddings?

Traditional encoding techniques like Bag of Words (BoW) and TF-IDF have several drawbacks:

- They produce **high-dimensional and sparse** vectors.
- They do not capture **semantic meaning** or **context**.
- They treat words like "king" and "queen" as unrelated.

To overcome these limitations, we use **word embeddings**, which generate **dense, low-dimensional vectors** where:

- Similar words have similar vectors
- Context and meaning are preserved
- Word relationships (like gender, geography, professions) can be understood through vector arithmetic

3.2.1 Word2Vec

What is Word2Vec?

Word2Vec is a two-layer shallow neural network that learns word embeddings using **local context** in a sentence.

It has two architectures:

Architecture	Description
CBOW (Continuous Bag of Words)	Predicts a center word from context
Skip-Gram	Predicts context words from the center word

CBOW (Continuous Bag of Words)

Concept

Given surrounding words, CBOW predicts the center word.

Example

Sentence: "We are learning Python"

If the context window is 2, then:

- Context: ["We", "are", "Python"]
- Target: "learning"

Architecture

1. Input: One-hot vectors for context words
2. Embedding: Shared embedding matrix maps each word to dense vector
3. Average: All context embeddings are averaged
4. Output: Softmax predicts the target word

Loss Function

$$\text{Loss}_{CBOW} = -\log P(w_t \mid w_{t-2}, w_{t-1}, w_{t+1}, w_{t+2})$$

Pros and Cons

Pros	Cons
Fast to train	Not effective for rare words
Works well for frequent words	Averages can lose nuance

Skip-Gram

Concept

Given a center word, Skip-Gram predicts surrounding context words.

Example

Sentence: "We are learning Python"

- Center word: "learning"
- Context: ["We", "are", "Python"]

Architecture

1. Input: One-hot of the center word

- 2. Embedding: Converts to dense vector
- 3. Output: Multiple softmax layers predict context words

Loss Function

$$\text{Loss}_{\text{SkipGram}} = - \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} \mid w_t)$$

Pros and Cons

Pros	Cons
Works well for rare words	Slower to train
Captures fine-grained relations	Needs more data

Comparison of CBOW vs. Skip-Gram

Feature	CBOW	Skip-Gram
Input	Context words	Center word
Output	Center word	Context words
Training Speed	Faster	Slower
Good For	Frequent words	Rare words
Prediction	One word	Multiple words

Code Example (Using Gensim)

1. Training Word2Vec Without Pretrained Model

Copy
Edit

```
python
from gensim.models import Word2Vec

sentences = [
    ["we", "are", "learning", "python"],
    ["python", "is", "used", "for", "data", "science"]
]

# CBOW
cbow_model = Word2Vec(sentences, vector_size=50, window=2, sg=0)
# Skip-Gram
skipgram_model = Word2Vec(sentences, vector_size=50, window=2, sg=1)

print("CBOW Vector:", cbow_model.wv["python"])
print("SkipGram Vector:", skipgram_model.wv["python"])
```

2. Using a Pretrained Word2Vec Model (Google News)

```
from gensim.models import Word2Vec, KeyedVectors

# Custom Training Sentences
sentences = [
    ["we", "are", "learning", "python"],
    ["python", "is", "used", "for", "data", "science"]
]

# Train CBOW model
cbow_model = Word2Vec(sentences, vector_size=50, window=2, sg=0) # sg >> Skip-Gram
print("CBOW Vector:", cbow_model.wv["python"])

# Train Skip-Gram model
skipgram_model = Word2Vec(sentences, vector_size=50, window=2, sg=1)
print("SkipGram Vector:", skipgram_model.wv["python"])

# Load Pretrained Word2Vec Model (Google News)
model_path = 'GoogleNews-vectors-negative300.bin' # Update path
pretrained_model = KeyedVectors.load_word2vec_format(model_path, binary=True)
print("Pretrained Vector (Google News):", pretrained_model["python"])
```

3.2.2 GloVe (Global Vectors)

What is GloVe?

GloVe (by Stanford) creates word embeddings by analyzing **global co-occurrence statistics** across the entire corpus, rather than predicting words like Word2Vec.

It builds a **co-occurrence matrix**, then uses matrix factorization to learn embeddings.

Key Idea

Words that occur in similar contexts should have similar embeddings.

Example: "ice" co-occurs more with "cold", while "steam" with "hot". GloVe uses such information to distinguish words.

Comparison to Word2Vec

Feature	Word2Vec	GloVe
Training	Predictive (context)	Count-based (global stats)
Context Type	Local	Global
Output Vectors	Learned from prediction	From matrix factorization

Code to Load GloVe Vectors

```
import numpy as np

def load_glove_model(file_path):
    embeddings = {}
    with open(file_path, 'r', encoding='utf8') as f:
        for line in f:
            parts = line.split()
            word = parts[0]
            vector = np.array(parts[1:], dtype='float32')
            embeddings[word] = vector
    return embeddings

glove = load_glove_model('glove.6B.100d.txt')
print(glove["python"])

Download GloVe vectors from: https://nlp.stanford.edu/projects/glove/
```

3.2.3 FastText

What is FastText?

FastText is developed by Facebook and improves Word2Vec by including **subword information**.

Instead of learning a single vector per word, it learns vectors for **subwords (n-grams)** and combines them.

Why is FastText Better?

- Handles **OOV (Out-of-Vocabulary)** words
- Useful for **morphologically rich languages**
- Better with **rare or misspelled words**

Example

"learning" is broken into:

- "lea", "ear", "arn", "rni", "ing" (character n-grams)

If "learnings" is not in the training data, it can still be represented using subword parts.

Code Example

```
from gensim.models import FastText

sentences = [
    ["we", "are", "learning", "python"],
    ["python", "is", "used", "for", "data", "science"]
]

model = FastText(sentences, vector_size=100, window=3, min_count=1)

print("Vector for 'python':", model.wv["python"])
print("Vector for unknown word 'pythoons':", model.wv["pythoons"])
```


3.2.4 Comparison Summary

Feature	Word2Vec	GloVe	FastText
Training Type	Predictive	Count-based	Predictive + Subword
Uses Context	Local	Global	Local + Subword
Handles OOV Words	No	No	Yes
Good For	General text	Global meaning	Rare words, spelling variants

◆ 4. Sequence Modeling Basics

4.1 Why Sequence Modeling in NLP?

In Natural Language Processing (NLP), the **order of words** plays a critical role in determining the meaning of a sentence. A change in word order can completely alter the context or interpretation.

Traditional machine learning models treat input features as **independent and unordered**, which may work for structured tabular data — but not for language. Text is inherently **sequential**, where each word’s meaning often depends on its position and its surrounding words.

Example

Consider the two sentences:

- “The dog chased the cat.”
- “The cat chased the dog.”

Both contain the same words, but the **sequence** changes the **subject and object**, completely altering the meaning.

4.2 Why Do We Need Specialized Models?

To effectively model sequential relationships, we need models that can:

- **Retain memory** of previously seen words
- Handle **variable-length sequences**
- Dynamically adjust to the context while processing

This is why we use **specialized deep learning models** designed for sequence data, such as:

- **Recurrent Neural Networks (RNNs)**
- **Long Short-Term Memory networks (LSTMs)**
- **Gated Recurrent Units (GRUs)**

These models process sequences one element at a time while maintaining a **hidden state** that carries information from earlier steps.

Summary

- Sequence modeling is essential in NLP, where **order and context** define meaning.
- Traditional models fall short because they ignore dependencies between elements in a sequence.
- Models like **RNN, LSTM, and GRU** are built to capture these dependencies and will be explored in detail in the next sections.

■ 5. Recurrent Neural Networks (RNNs)

1. 🔍 What is an RNN?

A **Recurrent Neural Network (RNN)** is a type of neural network designed for **sequential data**. Unlike traditional feedforward neural networks, RNNs maintain a **memory of previous inputs** by looping information through **hidden states** over time.

This makes RNNs highly effective for tasks where **order and context matter**, such as:

- Natural Language Processing (NLP)
- Time-series forecasting
- Speech recognition
- Music generation

🔄 Why “Recurrent”?

The term "recurrent" comes from the fact that the same operation is applied repeatedly at each time step, with the network reusing its previous output (hidden state) as part of the next input. This recurrence enables the network to "remember" past information as it processes a sequence

What is Sequential Data?

Sequential data is data where the **order of elements is important**. The current element often depends on what came before it. Examples include:

- Sentences (word order affects meaning)
- Stock prices over time
- Audio waveforms
- Video frames

Example:

"**We are learning data science in Velocity.**"

Sequential Aspect:

- The sentence makes sense **only if words appear in the correct order**.
- Changing the order to "Science are velocity data learning we" breaks the meaning.

In an RNN:

- At `t=1` , input = "We"
- At `t=2` , input = "are" + memory of "We"
- At `t=3` , input = "learning" + memory of "We are"
- And so on...

The RNN uses its **hidden state `h_t`** to accumulate understanding from left to right, capturing the full meaning as the sentence progresses.

What is Semantic Meaning?

Semantics deals with the **meaning of words** and their **relationships**. In our sentence:

Word	Semantic Role
"learning"	Implies education or knowledge
"data science"	A specific technical field
"Velocity"	A proper noun (name of institute), not the physics concept

RNNs handle semantics through **word embeddings** (e.g., Word2Vec, GloVe), which position similar-meaning words close together in vector space.

Why RNNs for Sequential + Semantic Understanding?

RNNs are powerful because they:

-  **Maintain sequence:** They process one word at a time, keeping track of past inputs.
-  **Understand context:** Using hidden states, they can learn **how previous words influence the meaning** of the current word.
-  **Capture semantics:** Through embeddings and context, they infer word meanings accurately.

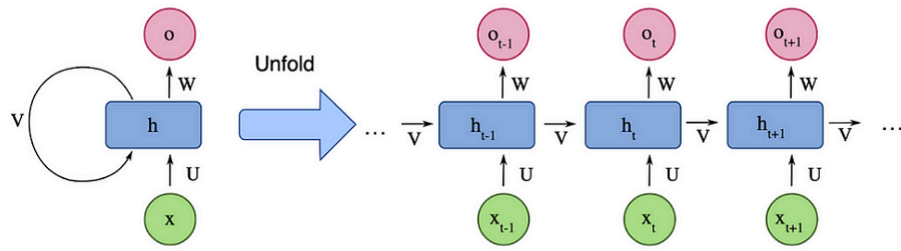
So in the sentence "We are learning data science in Velocity." , RNNs can:

- Understand it's about **education/training**,
- Recognize "Velocity" as an **organization**,
- And use word positions to **classify, translate, or summarize** the sentence properly.

2. RNN vs Feedforward Neural Network

Feature	Feedforward NN	RNN
Data Type	Fixed-size input	Sequential input
Memory of Past Inputs	 No	 Yes
Shared Weights Over Time	 No	 Yes
Output Depends On	Current input only	Current + previous inputs
Ideal For	Images, tabular data	Text, time-series, speech

3. RNN Architecture



RNN Flow Chart

3.1 How RNN Works at Each Time Step

At every time step t , the RNN updates its internal memory (hidden state) and optionally produces an output. The key equations are:

✓ Core Equations:

1. Hidden state computation:

$$h_t = \tanh(Wx_t + Uh_{t-1} + b)$$

2. Output computation:

$$y_t = Vh_t + c$$

✚ Explanation of Each Term

1 x_t – Input at time step t

- The **current word, character, or time-series value**.
- Example: In the sentence "I love cats", at $t=2$, x_2 represents "love" (as an embedding or one-hot vector).

2 h_{t-1} – Previous hidden state

- Stores **memory** from previous steps.
- Summarizes all inputs up to time $t-1$.

What about the first time step ($t = 0$)?

- h_0 is manually initialized, usually to a **zero vector**:

$$h_0 = \vec{0}$$

- Shape: `(batch_size, hidden_size)`

Some advanced models use random or trainable h_0 , but zero is most common and stable.

3 W – Input-to-hidden weight matrix

- Transforms input x_t into hidden space.
- Shape: `(hidden_size, input_size)`

4 U – Hidden-to-hidden (recurrent) weights

- Connects past hidden state h_{t-1} to current h_t .
- Shape: `(hidden_size, hidden_size)`

5 b – Hidden bias

- Added to improve flexibility in learning.
- Shape: `(hidden_size,)`

6 $\tanh()$ – Activation Function

- Introduces **non-linearity**.
- Squashes output to range `[-1, 1]`.

Prevents values from growing too large and helps RNNs learn better patterns.

7 h_t – Current hidden state

- The RNN's **internal memory** at time t .

- Combines the influence of `x_t` and `h_{t-1}`.
- Passed to next step and optionally to output.

8 **V – Hidden-to-output weight matrix**

- Converts hidden state `h_t` to raw output `y_t`.
- Shape: `(output_size, hidden_size)`

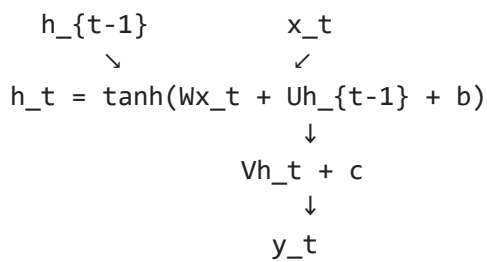
9 **c – Output bias**

- Added to the output for flexibility.
- Shape: `(output_size,)`

 y_t – Output at time step t

- Final output from the current step.
- Depends on the task:
 - **Classification**: passed through `softmax` or `sigmoid`
 - **Regression**: used as-is

 Summary of Data Flow at Time Step t



3.2 RNN Parameter Sharing & Input Types

 Parameter Sharing in RNNs

- The weights `W`, `U`, `V`, and biases `b`, `c` are **shared across all time steps**.
- This reduces the number of parameters and allows the model to generalize across sequences of varying length.

 Key Takeaway

At each time step:

- The RNN **takes the current input** and **remembers the past via** `h_{t-1}`,
- **Processes them with shared weights**,
- **Updates its memory** (`h_t`), and optionally **produces an output** (`y_t`).

This loop gives RNNs the power to **"remember"** past information in a sequence.

 Input Types for RNNs and the Role of Embedding Layer

Input Type	Vectorized?	Dense/Sparse	Semantic Meaning?	Embedding Layer Required?	Passable to RNN Directly?	Trainable?	Use Case
Integer Encoding	✗ No	N/A	✗ No	✓ Yes	✗ No	✓ Yes	Most common with embedding
One-Hot Encoding	✓ Yes	✗ Sparse	✗ No	✗ No	✓ Yes	✗ No	Rarely used
TF-IDF Vector	✓ Yes	Mixed	✗ Frequency only	✗ No	✓ Yes	✓ Yes	Text classification
Pretrained Word2Vec	✓ Yes	✓ Dense	✓ Yes	✗ No	✓ Yes	✗ No	Semantic input
Pretrained GloVe	✓ Yes	✓ Dense	✓ Yes	✗ No	✓ Yes	✗ No	NLP tasks
Pretrained FastText	✓ Yes	✓ Dense	✓ Yes (with subwords)	✗ No	✓ Yes	✗ No	Rare words, multilingual NLP
Trainable Embedding Layer	✓ Yes (learned)	✓ Dense	✓ Learns meaning	✓ Yes	✓ Yes	✓ Yes	Best for task-specific embeddings

 Summary:

- **RNNs expect dense vectorized inputs** → most workflows use an **embedding layer** to convert integer-based tokens into learnable word vectors.
- **Pretrained embeddings** like GloVe or Word2Vec **carry semantic relationships** and can be passed directly to the RNN.

4. Backpropagation Through Time (BPTT)

What is BPTT?

Backpropagation Through Time (BPTT) is the training algorithm used for **Recurrent Neural Networks (RNNs)**. It is an extension of regular backpropagation that allows RNNs to **learn from sequences** by computing gradients **across all time steps**.

Why Not Standard Backpropagation?

In RNNs, outputs at each time step depend on **previous hidden states**, which introduces **temporal dependencies**. To account for this, the RNN is **unrolled over time**, and gradients are calculated **across all steps** → hence the term **“through time.”**

Forward Pass (Per Timestep)

Given input sequence `x0, x1, ..., xT`, the RNN computes:

$$h_t = \tanh(Wx_t + Uh_{t-1} + b) \quad y_t = Vh_t + c$$

Where:

- `W` = input-to-hidden weights
- `U` = hidden-to-hidden (recurrent) weights
- `V` = hidden-to-output weights
- `b`, `c` = biases

These weights are **shared across time steps**.

Loss Computation

The total loss over the sequence is:

$$L = \sum_{t=1}^T \mathcal{L}(y_t, \hat{y}_t)$$

Backward Pass (BPTT)

Step-by-Step:

1. **Unroll the RNN** across all time steps (like a feedforward network over time).
2. Compute the **loss gradient** w.r.t. outputs `yt` at each time step.
3. Backpropagate through time:
 - From time `T` down to time `1`
 - Accumulate gradients for **all shared weights**
4. Update parameters using gradient descent.

Weights That Get Updated

Parameter	Role	Shared across time?	Updated in BPTT?
<code>W</code>	Input to hidden	✓ Yes	✓ Yes
<code>U</code>	Hidden to hidden (recurrence)	✓ Yes	✓ Yes
<code>V</code>	Hidden to output	✓ Yes	✓ Yes
<code>b</code>	Hidden layer bias	✓ Yes	✓ Yes
<code>c</code>	Output layer bias	✓ Yes	✓ Yes

Gradient Behavior in BPTT

Vanishing Gradient:

- Gradients **shrink** over time steps.
- Makes it hard to learn **long-term dependencies**.
- Affects weight `U` the most (recurrent connection).

▲ Exploding Gradient:

- Gradients **grow uncontrollably**.
- Causes unstable weight updates.
- Solved using **gradient clipping**.

✂ Truncated BPTT (Practical Variant)

Instead of backpropagating through **all time steps**, we can limit it to a fixed number of steps `k`.

Method	Description
Full BPTT	Backprop through entire sequence
Truncated BPTT	Backprop through only <code>k</code> recent steps (e.g., 10)

Reduces computational load and memory usage. Common in long sequence training.

✓ Summary

- BPTT is required to train RNNs because they have **recurrent connections** over time.
- The RNN is **unrolled**, and gradients are computed through **all time steps**.
- Weights `W`, `U`, `V`, `b`, and `c` are **shared across timesteps** and **updated using the sum of gradients** from all steps.
- Use **truncated BPTT** and **gradient clipping** to manage long sequences and instability.

5. 🧩 Types of RNN Input-Output Structures

Type	Input	Output	Examples
One-to-Many	One input	Sequence output	Text generation from a seed
Many-to-One	Sequence input	One output	Sentiment analysis
Many-to-Many (sync)	Sequence input	Sequence output (same size)	POS tagging, NER
Many-to-Many (async)	Sequence input	Sequence output (diff size)	Machine translation

6. ⚙ RNN Implementation in Keras

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

model = Sequential()
model.add(Embedding(input_dim=5000, output_dim=64)) # word vectors
model.add(SimpleRNN(units=32)) # hidden state
model.add(Dense(1, activation='sigmoid')) # output

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

7. ⚠ Limitations of Vanilla RNN

✗ Vanishing Gradient

- As gradients backpropagate through time, they shrink and **fail to update earlier layers**.

✗ Exploding Gradient

- Gradients can also grow too large, making the model **unstable**.

✗ Short-Term Memory

- RNNs forget long sequences due to poor memory control.

✗ Slow Training

- Computation is **sequential**, making training less parallelizable.

8. Effective Sequence Length

Model	Effective Words (Timesteps)	Notes
RNN	10–20	Suffers from vanishing gradients
LSTM	100–200	Retains long-term dependencies
GRU	80–150	Better than RNN, simpler than LSTM

9. Variants of RNN

◆ Functional Variants (RNN Cell Types)

These modify **how a single time step is processed** internally.

Variant	Description
Vanilla RNN	Basic model with <code>tanh</code> activation
LSTM	Adds memory cell and 3 gates (input, forget, output)
GRU	Simplified LSTM with 2 gates (update, reset)

◆ Architectural Variants (RNN Structures)

These modify **how layers are organized** or how input is processed.

Variant	Description
Stacked RNN	Multiple RNN layers stacked for depth
Bidirectional RNN	Processes input forward and backward
Stacked Bidirectional RNN	Combines both stacking and bi-directionality

✔ These architectural changes enhance depth or context but do **not change the internal logic** of how a time step is processed.

10. Deep RNN Variants

Deep RNNs are enhanced architectures designed to learn **complex, hierarchical, and bidirectional dependencies** in sequential data. These variants include **stacked, bidirectional**, and their **combined** forms.

10.1 Stacked RNN

✚ Definition:

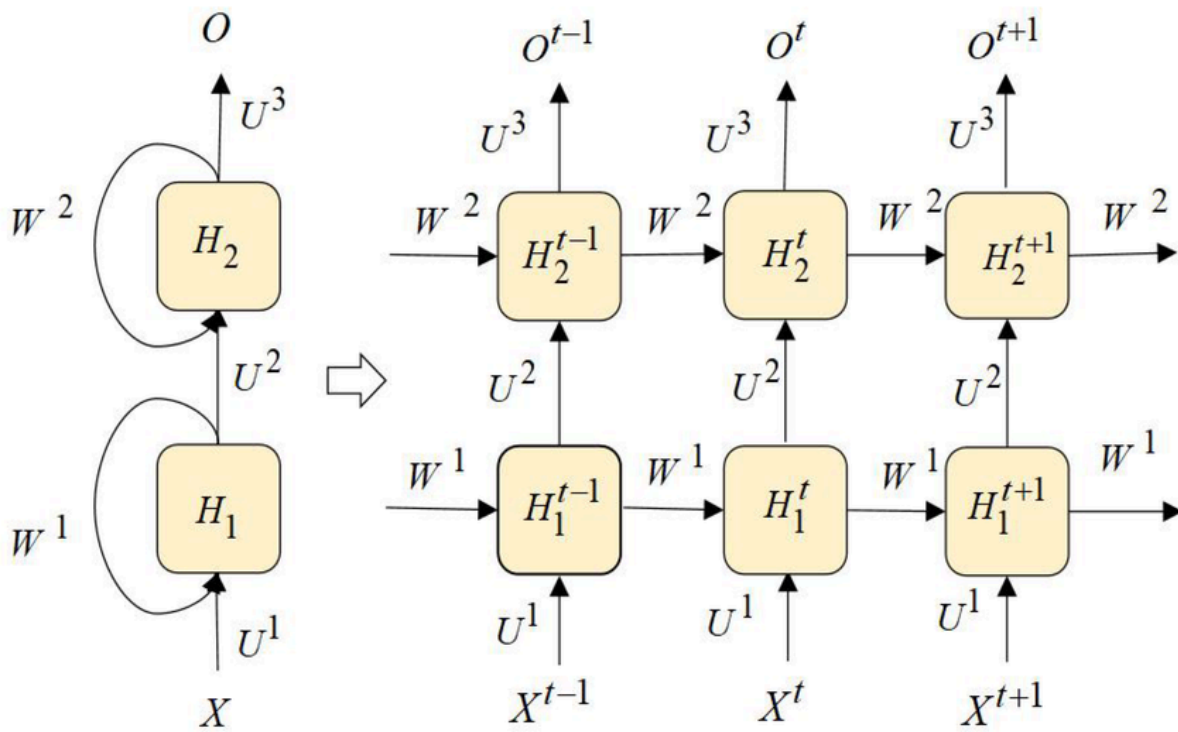
A **Stacked RNN** (also called a multi-layer RNN) is a deep architecture where **multiple RNN layers are stacked on top of each other**, with each layer passing its hidden state output to the next layer at the **same time step**.

Depth in time (across time steps) + **Depth in layers** (across vertical layers)

Why Stack RNN Layers?

While a single-layer RNN can model sequential dependencies, stacking multiple RNN layers allows the network to learn **hierarchical representations**:

Layer	Learns What?
Layer 1	Local patterns (e.g., word-level)
Layer 2	Phrase-level or sentence-level patterns
Layer 3	High-level abstract features



🔧 Architecture

Let's say we have an input sequence $x = [x_1, x_2, \dots, x_T]$ and L stacked layers:

◆ First Layer:

$$h_t^{(1)} = f(W^{(1)}x_t + U^{(1)}h_{t-1}^{(1)} + b^{(1)})$$

◆ Second Layer:

$$h_t^{(2)} = f(W^{(2)}h_t^{(1)} + U^{(2)}h_{t-1}^{(2)} + b^{(2)})$$

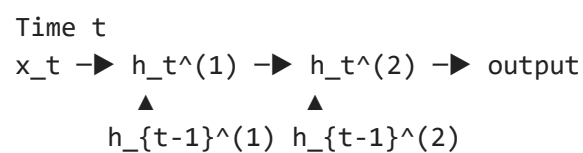
... and so on for each layer 1 .

Each layer receives:

- Input from the **previous layer at the same time step**
- Hidden state from the **previous time step in the same layer**

📊 Flow Summary

For 2-layer Stacked RNN:



🧩 Keras Example

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
```

```
model = Sequential([
    Embedding(input_dim=10000, output_dim=64, input_length=100),
    SimpleRNN(128, return_sequences=True), # Layer 1
    SimpleRNN(64, return_sequences=True), # Layer 2
    SimpleRNN(32),                        # Layer 3
    Dense(1, activation='sigmoid')
])
```

🗣️ Real-Time Example:

Sentence: "We are learning NLP"

Let's say we're doing sequence classification (e.g., classify the sentence as TECH-related).

- **Layer 1:** Learns short-term dependencies like "We are", "learning NLP"
- **Layer 2:** Captures the grammatical flow of the entire sentence
- **Layer 3:** Recognizes global meaning → tech context

The final hidden state from Layer 3 is used for output.

✔ Benefits of Stacked RNN

Benefit	Description
✔ Hierarchical abstraction	Learns features at different complexity levels
✔ Deeper temporal modeling	Can handle long sequences better
✔ Higher capacity	Learns more complex patterns
✔ Flexibility	Can tune number of layers, units per layer

⚠ Limitations of Stacked RNN

Limitation	Description
✖ Vanishing gradients	Gets worse with more layers (better with LSTM/GRU)
⚠ Overfitting risk	More parameters need regularization (e.g., Dropout)
⚠ Slower training	Especially for long sequences and deep stacks
⚠ Careful tuning required	Number of layers, units, learning rate, etc.

🔄 Output Control

In Keras or PyTorch:

- **Intermediate layers** must use `return_sequences=True`
- **Final layer** can use:
 - `return_sequences=False` → for sequence-to-label
 - `return_sequences=True` → for sequence-to-sequence

📊 Ideal Use Cases for Stacked RNN

- ✔ Speech recognition
- ✔ Machine translation (encoder side)
- ✔ Sentiment classification
- ✔ Time-series forecasting
- ✔ Complex NLP sequences

🔄 Stacked RNN vs Single-Layer RNN

Feature	Single RNN	Stacked RNN
Depth of understanding	Low	High
Pattern complexity	Simple	Complex
Context range	Limited	Broader (longer memory)
Overfitting risk	Lower	Higher (if not regularized)

📄 Summary Table

Term	Description
$h_t^{(1)}$	Hidden state from layer 1
$h_t^{(2)}$	Hidden state from layer 2
x_t	Input at time step t
<code>return_sequences=True</code>	Pass sequence to next layer
<code>return_sequences=False</code>	Return only last hidden state
Stacked flow	Output of one layer → input to next layer at same time step

✔ Final Takeaway

Stacked RNNs enable **deep sequential learning** by building layer-by-layer understanding of temporal patterns — from basic to abstract — making them ideal for **complex language, audio, and time-series tasks**.

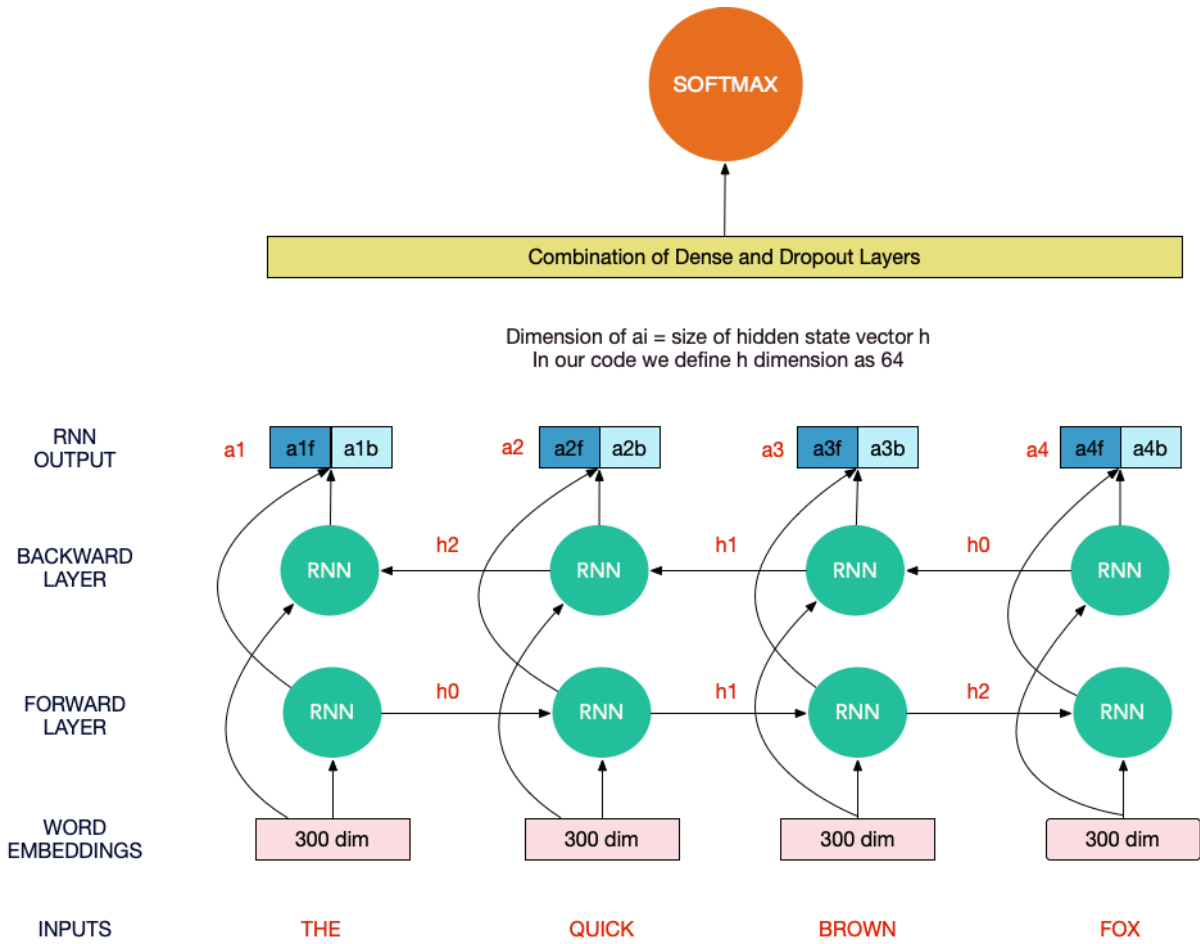
10.2 Bidirectional RNN

A **Bidirectional Recurrent Neural Network (BiRNN)** processes input sequences in **both directions**:

- One RNN processes from **left to right** (past → future)
- Another RNN processes from **right to left** (future → past)

At each time step, the hidden states from both directions are **combined** (typically concatenated) to produce a richer representation.

Architecture



For input sequence $x = [x_1, x_2, \dots, x_T]$:

◆ Forward RNN:

$$\vec{h}_t = f(U_f x_t + V_f \vec{h}_{t-1} + b_f)$$

◆ Backward RNN:

$$\overleftarrow{h}_t = f(U_b x_t + V_b \overleftarrow{h}_{t+1} + b_b)$$

◆ Final Combined Representation:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \quad \text{or} \quad h_t = \vec{h}_t + \overleftarrow{h}_t$$

This combined state is used for output:

$$o_t = \text{Dense}(h_t)$$

Why Bidirectional?

Standard (unidirectional) RNNs only consider **past context**, which limits understanding in many tasks. BiRNNs use **both past and future context**, leading to **more accurate predictions**.

Real-Time Understanding Examples

◆ Example 1: "We are learning NLP."

- **Word of interest:** "learning"

Direction	What it sees	Inference
Forward RNN	"We are"	Maybe verb or noun
Backward RNN	"NLP" (from future)	Confirms it's a verb
BiRNN Result	Combined context	✓ Correct: Verb (VBG)

◆ **Example 2: "Apple is looking to expand in India."**

- **Word of interest:** "Apple"

Direction	What it sees	Inference
Forward RNN	"Apple is looking"	Maybe fruit or company
Backward RNN	"to expand in India"	Suggests company
BiRNN Result	Combined context	✔ Correct: ORGANIZATION

◆ **Example 3: "She made a race today."**

- **Word of interest:** "race"

Direction	What it sees	Inference
Forward RNN	"She made a"	Possibly a verb
Backward RNN	"today"	Implies it's a noun
BiRNN Result	Combined context	✔ Correct: Noun (event)

✔ **Benefits of BiRNN**

Benefit	Description
✔ Past + Future context	Improves understanding and accuracy
✔ Better disambiguation	Resolves meaning of ambiguous words
✔ Richer feature representation	Concatenated hidden states capture more signal
✔ Improved performance	In POS tagging, NER, sentiment, translation, etc.

⚠ **Limitations of BiRNN**

Limitation	Description
✗ Not usable in real-time	Needs full input sequence in advance
⚠ Double computation	Two RNNs (forward and backward) double parameters
⚠ Slower inference	Especially for long sequences

🔄 **When Forward and Backward RNNs Interact**

Stage	Interaction?	What Happens
During processing	✗ No	Each RNN runs independently
After full sequence seen	✔ Yes	Hidden states are combined at each time step
Final output layer	✔ Yes	Output is based on <code>[h_t^→ ; h_t^←]</code>

📊 **Ideal Use Cases for BiRNN**

- ✔ Named Entity Recognition (NER)
- ✔ Part-of-Speech (POS) Tagging
- ✔ Sentiment Analysis
- ✔ Document Classification
- ✔ Text Summarization (Encoder side)
- ✔ Speech Recognition

📄 **Summary Table**

Feature	Unidirectional RNN	Bidirectional RNN
Past context	✔ Yes	✔ Yes
Future context	✗ No	✔ Yes
Parallel RNNs	✗ One	✔ Two
Output per word	Based on past only	Based on full context
Real-time friendly	✔ Yes	✗ No
Accuracy (complex tasks)	Lower	✔ Higher

✓ Final Takeaway

Bidirectional RNNs enhance performance by giving each word access to **both previous and upcoming context**, making them ideal for **NLP tasks** where **meaning depends on full sequence understanding**.

10.3 🔄 Stacked + Bidirectional RNN

📌 Definition:

A **Stacked Bidirectional RNN** combines both stacking and bidirectionality. Each layer is bidirectional, and multiple such layers are stacked. This captures **multi-level features** with **full context** from both directions.

💡 Key Concepts:

Feature	Description
Depth + Bidirectionality	Each layer is bidirectional; layers are stacked
Output at each layer	Concatenated forward and backward hidden states
Strong Feature Learning	Best for complex, long, and structured sequences

📄 Layer-wise Output:

- First Layer:

$$h_t^{(1)} = [\overrightarrow{h_t^{(1)}}; \overleftarrow{h_t^{(1)}}]$$

- Second Layer:

$$h_t^{(2)} = [\overrightarrow{h_t^{(2)}}; \overleftarrow{h_t^{(2)}}]$$

where each direction receives $h_t^{(1)}$ as input.

🔗 Keras Example:

```
model = Sequential([
    Embedding(input_dim=10000, output_dim=64, input_length=100),
    Bidirectional(SimpleRNN(128, return_sequences=True)), # Layer 1
    Bidirectional(SimpleRNN(64)), # Layer 2
    Dense(1, activation='sigmoid')
])
```

✓ When to Use:

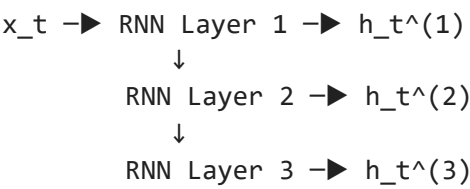
- Document classification
- Speech recognition
- Machine translation

⚠️ Limitations:

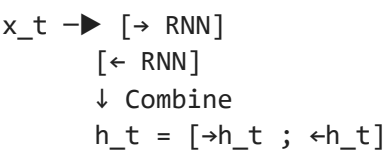
- Expensive in memory and compute
- More prone to overfitting
- Not suitable for real-time

📊 Diagram Summary

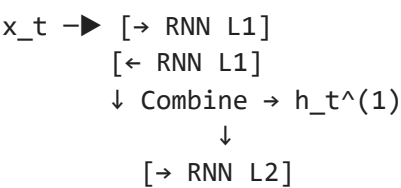
✓ A. Stacked RNN



✓ B. Bidirectional RNN



✓ C. Stacked Bidirectional RNN



11. 🔄 Comparison between RNN Variants: RNN (Vanilla) vs Stacked RNN vs Stacked + Bi-RNN

🔍 Feature	🔵 RNN (Vanilla)	🟢 Stacked RNN	🟡 Bi-RNN	🔴 Stacked + Bi-RNN
📄 Context used	Past only	Past (multi-layer)	Past + Future	Past + Future (multi-layer)
💡 Word representation	Basic	Deeper, layered	Rich (bi-context)	Deep + Rich (bi + hierarchical)
🧠 Effective understanding	❌ Partial	✅ Moderate	✅✅ Strong	✅✅✅ Excellent
🔍 Ambiguity resolution	Weak	Better than RNN	Strong	Very strong
🏗️ Feature hierarchy per word	1 Layer	2+ Layers	1 Layer (bi)	2+ Layers (bi)
🌱 Each word sees...	~2–4 previous words	~6–10 past words	~4–8 before + 4–8 after	~10–20 full context words
📊 Effective number of words	~2–4	~6–10	~8–16	~20+
🔧 Handles how many total words?	~20–30 before vanishing gradient	~40–60 words (deeper memory)	~30–40 words (bi adds cost)	~50–80 words (best performance range)
💡 Word vector quality	🟠 Low	🟡 Moderate	🟢 High	🟢🟢 Very High
📈 Performance in word-level tasks	Lower	Moderate	High	Best
📄 Word meaning = function of...	Past context only	Past (deeper layers)	Past + Future	Past + Future + Depth

12. 📦 Real-World Applications

Domain / Use Case	Description
🧠 Language Modeling	Predict the next word in a sentence (e.g., GPT training)
🔥 Text Generation	Generate text like poems, stories, or code
🗣️ Speech Recognition	Convert spoken audio into written text
🌐 Machine Translation	Translate sentences between languages using sequence-to-sequence models
📈 Stock / Time-Series Prediction	Forecast future values based on past data
😊 Sentiment Analysis	Identify sentiment (positive, negative, neutral) in text
📄 Named Entity Recognition (NER)	Extract names, organizations, dates, etc. from text
🏷️ Text Classification	Assign categories to text (e.g., spam vs. ham, topic tagging, intent detection)

13. 🛡️ Techniques to Mitigate Gradient Problems

Problem	Solution
Vanishing Gradient	Use LSTM or GRU
Exploding Gradient	Apply gradient clipping
Slow Training	Use efficient variants (GRU)
Overfitting	Apply dropout, regularization

14. 🧠 Summary

- RNNs are powerful for **sequential data** but suffer from training challenges.
- For **long-term memory**, use **LSTM or GRU**.
- For richer context, use **bidirectional or stacked** architectures.
- Always analyze **sequence length, task complexity, and performance needs** before choosing the model.

📘 6. LSTM – Long Short-Term Memory

◆ 6.1 Why LSTM?

Problem with RNNs: They struggle to learn **long-term dependencies** due to the **vanishing gradient problem**.

Solution: LSTM introduces a more sophisticated memory mechanism using **gates** to control the flow of information and retain relevant context over long sequences.

◆ 6.2 What is an LSTM?

An **LSTM (Long Short-Term Memory)** is an enhanced RNN architecture that uses a **cell state** and **gates** to regulate memory.

It can **remember or forget** information selectively — allowing it to learn dependencies over long distances.

◆ 6.3 LSTM Example: "We are learning NLP"

Let’s say:

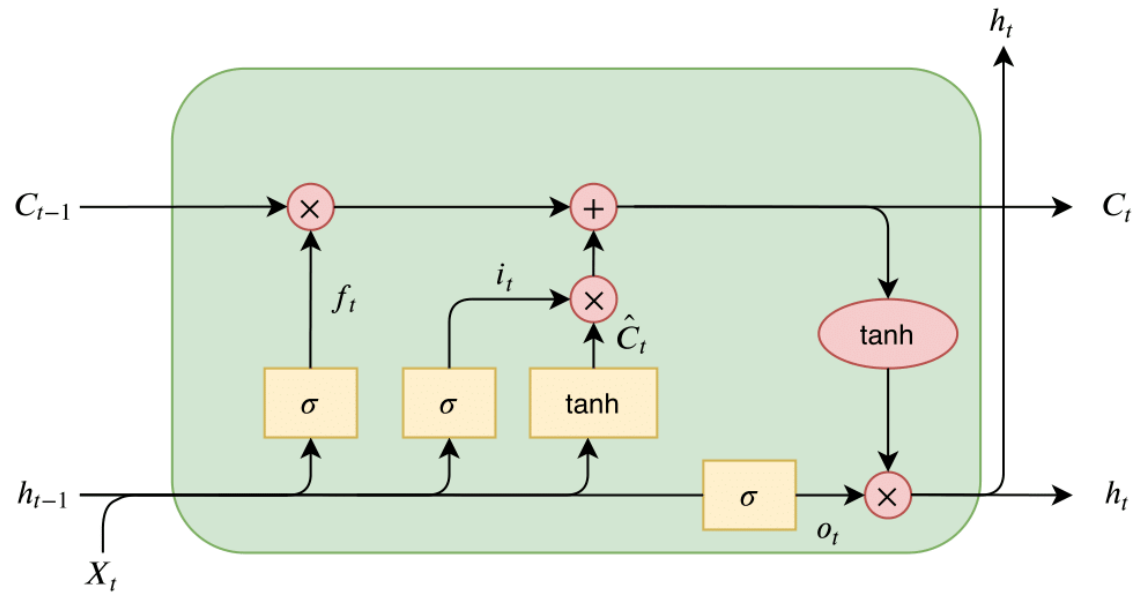
- We want to analyze sentiment or predict the next word.
- We process "We → are → learning → NLP" sequentially.

RNN might forget "We" by the time it reaches "NLP", but **LSTM can remember it**.

◆ 6.4 LSTM Architecture

Each LSTM unit has:

Component	Purpose
Cell state (C_t)	Memory of the network
Hidden state (h_t)	Output at that time step
Forget gate (f_t)	Decides what to forget
Input gate (i_t)	Decides what new info to store
Output gate (o_t)	Decides what to output



📊 Gate Functions

Let x_t = input at time t Let h_{t-1} = previous hidden state

1. Forget Gate:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

👉 Decides what part of previous cell state to forget

2. Input Gate:

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

👉 Decides what new info to add

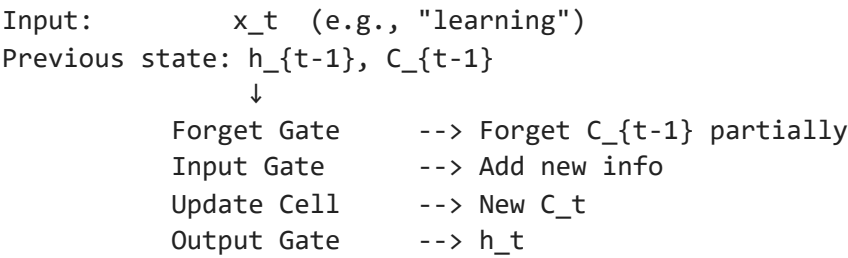
3. Update Cell State:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

4. Output Gate:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$
$$h_t = o_t * \tanh(C_t)$$

📦 Visual Flow per Time Step:



◆ 6.5 Applying LSTM on: "We are learning NLP"

t	Word	Input x _t	Cell State C _t	Hidden h _t
1	"We"	x ₁	Updated via forget+input	h ₁ output
2	"are"	x ₂	Carries memory from "We"	h ₂ (contextualized)
3	"learning"	x ₃	Builds stronger context	h ₃ (deep context)
4	"NLP"	x ₄	Final full context	h ₄ → used to predict

◆ 6.6 Code Example: LSTM in Keras

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential()
model.add(Embedding(input_dim=5000, output_dim=64, input_length=4)) # input: "We are Learning NLP"
model.add(LSTM(128, return_sequences=False)) # One output after full sequence
model.add(Dense(1, activation='sigmoid')) # Binary output (e.g., sentiment)
model.summary()
Use return_sequences=True if you want output at every time step.
```

◆ 6.7 LSTM Variants

Variant	Description
Bidirectional LSTM	Processes sequence in both directions
Stacked LSTM	Multiple LSTM layers stacked vertically
LSTM + Attention	Focuses on relevant parts of the input when predicting output

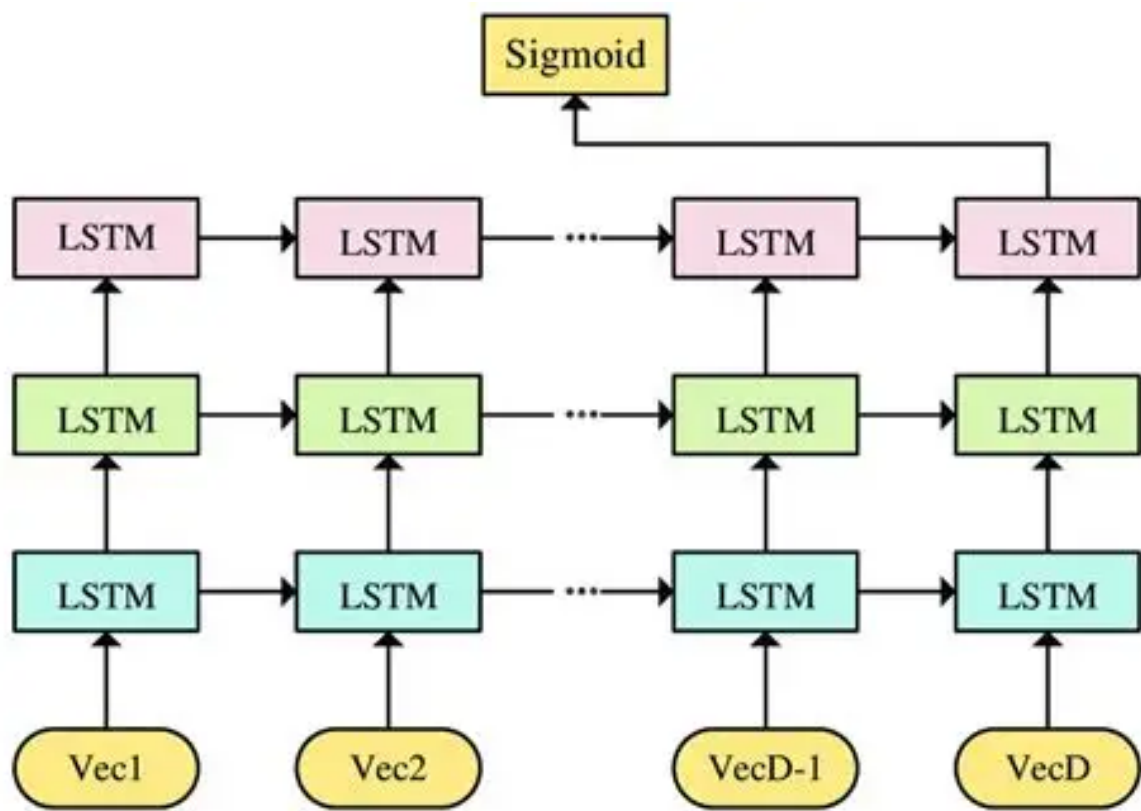
◆ 6.8 LSTM vs RNN

Feature	RNN	LSTM
Memory Length	Short	Long (thanks to gates)
Gradient Flow	Vanishing gradient problem	Preserves gradients
Structure	Simple	Complex (gates)
Performance	Struggles with long sequences	Handles long dependencies

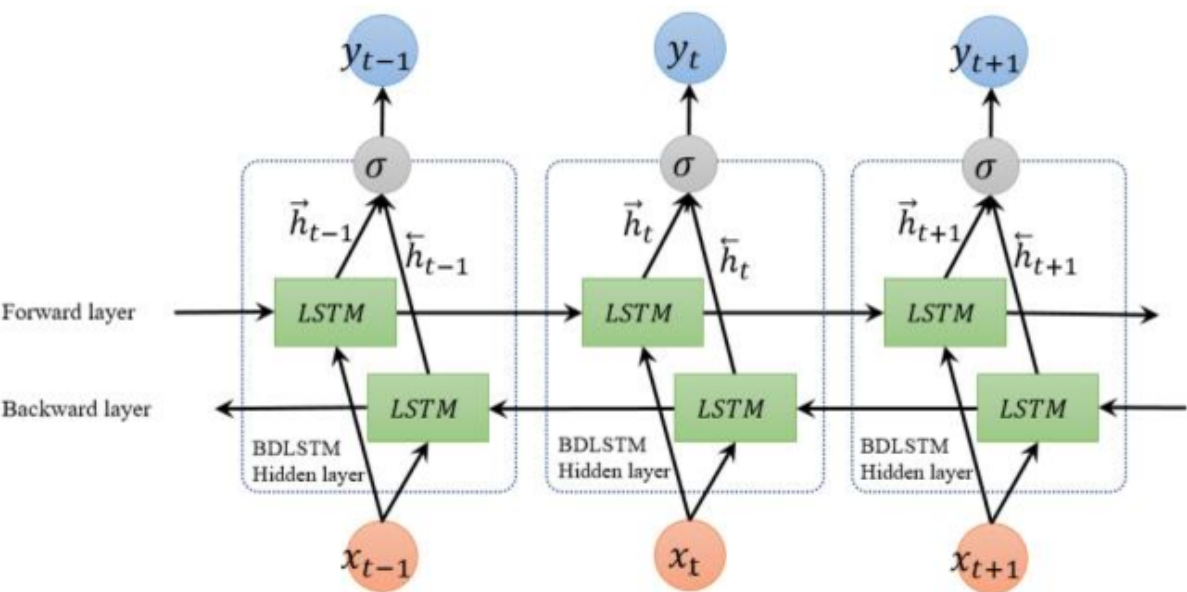
✅ Summary

- LSTM solves RNN’s limitation by adding **memory cells** and **gates**.
- It **remembers what’s important** and forgets irrelevant parts.
- Best suited for tasks like:
 - Sentiment analysis
 - Machine translation
 - Text summarization
 - Time series forecasting

Stacked LSTM



Bidirectional LSTM



7: GRU – Gated Recurrent Unit

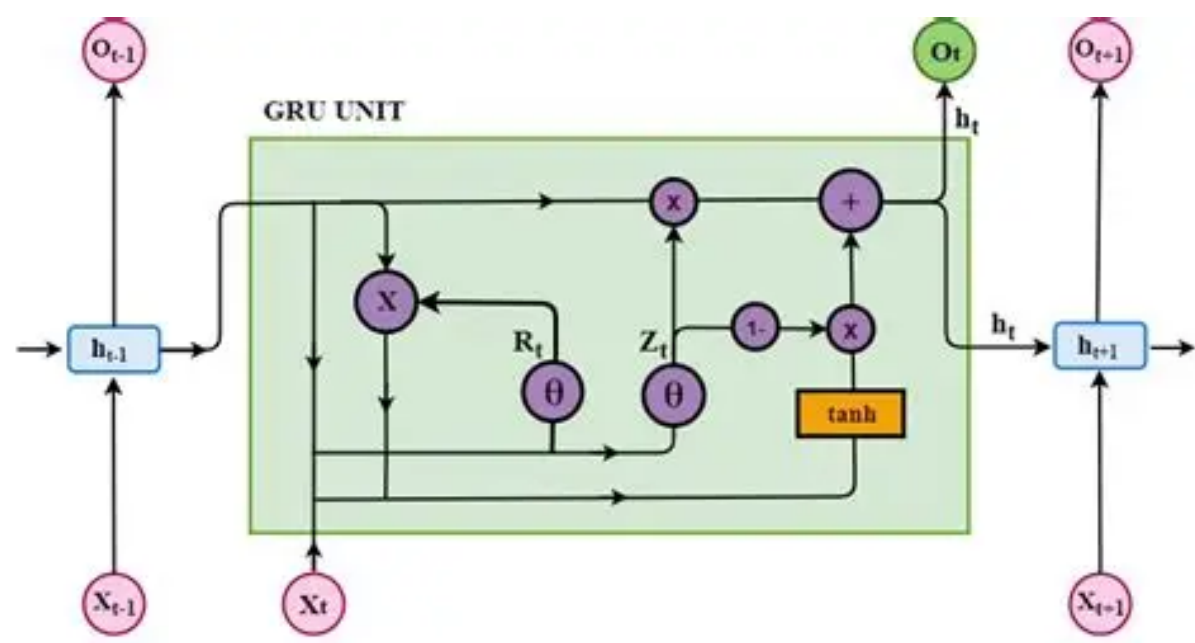
7.1 What is a GRU?

GRU (Gated Recurrent Unit) is a simplified version of **LSTM**, introduced by Cho et al. (2014). It retains most of LSTM's power but with fewer gates and no separate cell state — making it **faster to train** and **easier to implement**.

7.2 Why Use GRU?

Challenge in RNNs	Solution
Short-term memory	GRU keeps long dependencies
Too many parameters in LSTM	GRU simplifies architecture

7.3 GRU Architecture



GRU uses **two gates**:

Gate	Role
Update gate (z_t)	Controls how much of the previous memory to keep
Reset gate (r_t)	Controls how much of the past to forget

Unlike LSTM:

- GRU **combines cell state and hidden state** into one vector h_t
- No output gate — simpler and faster

GRU Equations

Let:

- x_t : input at time t
- h_{t-1} : previous hidden state

1. **Update gate:**

$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

2. **Reset gate:**

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

3. **Candidate activation:**

$$\tilde{h}_t = \tanh(W_h \cdot [r_t * h_{t-1}, x_t])$$

4. **Final hidden state:**

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

◆ 7.4 GRU vs LSTM (Key Differences)

Feature	LSTM	GRU
Gates	3 (input, forget, output)	2 (update, reset)
Memory	Cell state + hidden state	Only hidden state
Speed	Slower (more parameters)	Faster (simpler structure)
Performance	Slightly better on some tasks	Comparable, esp. on smaller data
Complexity	More complex	Simpler

◆ 7.5 Example: "We are learning NLP"

Suppose we're training a GRU to understand sentiment in this sequence.

t	Word	Hidden State h_t
1	"We"	h_1 (depends on x_1 and h_0)
2	"are"	h_2
3	"learning"	h_3

t	Word	Hidden State h_t
4	"NLP"	$h_4 \rightarrow$ used for final output

Each h_t summarizes the sentence up to that point.

◆ 7.6 Code Example: GRU in Keras

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense

model = Sequential()
model.add(Embedding(input_dim=5000, output_dim=64, input_length=4)) # e.g., "We are Learning NLP"
model.add(GRU(128))
model.add(Dense(1, activation='sigmoid')) # e.g., sentiment classification
model.summary()

To get output at every step:

model.add(GRU(128, return_sequences=True))
```

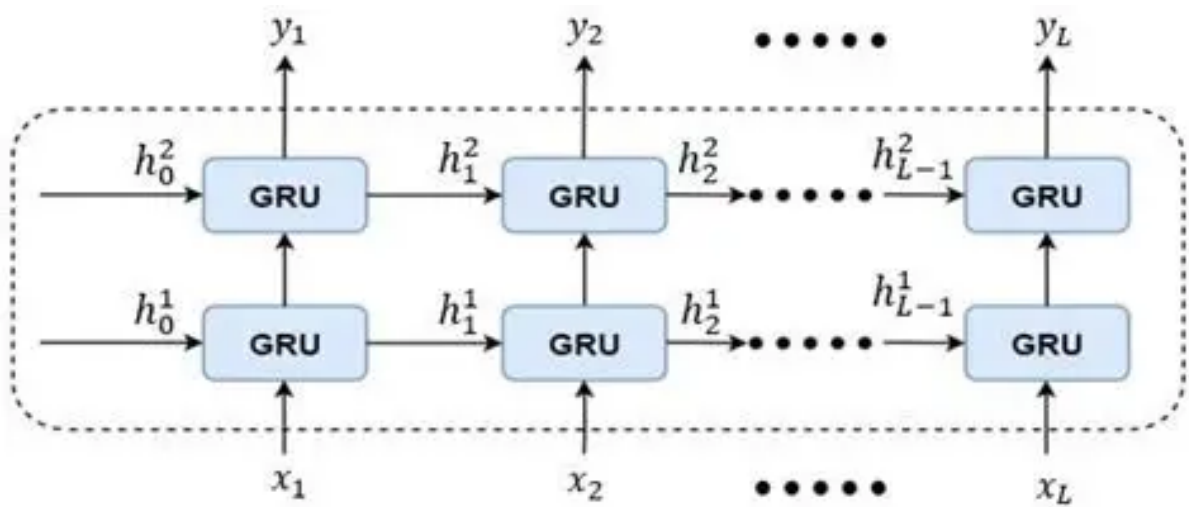
◆ 7.7 When to Use GRU

Use Case	LSTM or GRU?
Long sequences with strong dependencies	LSTM
Real-time applications (faster training)	GRU
Smaller datasets	GRU (less overfitting)
Large datasets, more expressiveness	LSTM

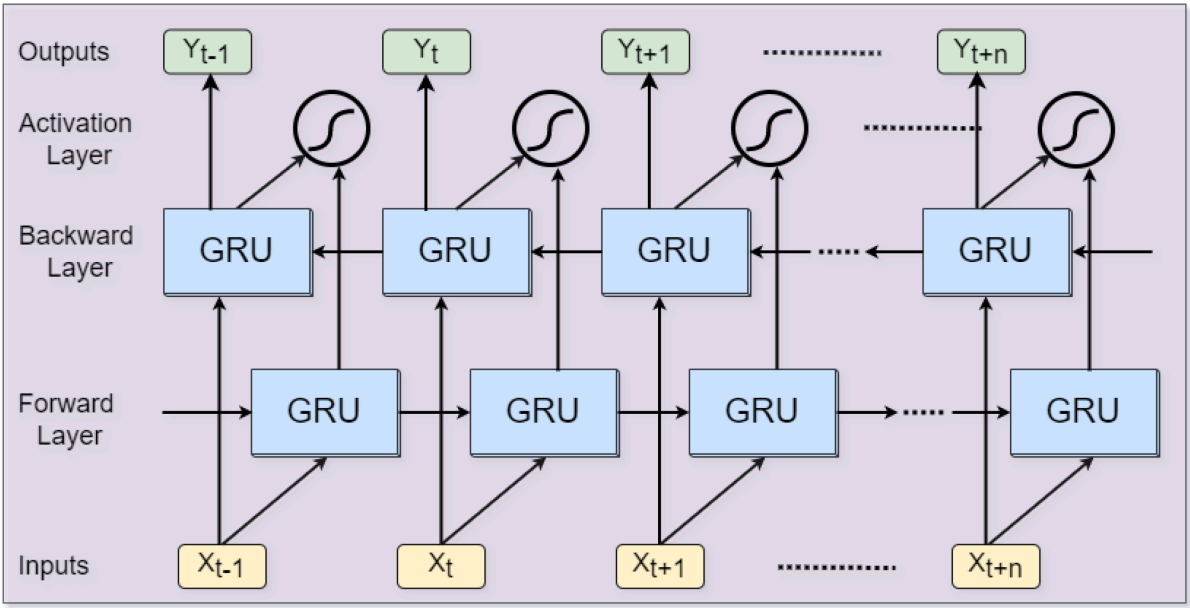
✓ Summary

- GRU is a **lighter, faster alternative** to LSTM.
- Works well when:
 - You need fast convergence
 - Memory or latency is a constraint
- Ideal for **chatbots, mobile apps, short-text analysis**

Stacked GRU



Bidirectional GRU



8. Comparison: RNN vs. LSTM vs. GRU

 **Table 1: Core Comparison – RNN vs. LSTM vs. GRU**

This table helps you understand the architectural and performance differences between the three core sequential models.

Feature / Aspect	RNN	LSTM	GRU
Full Form	Recurrent Neural Network	Long Short-Term Memory	Gated Recurrent Unit
Introduced By	Elman (1990s)	Hochreiter & Schmidhuber (1997)	Cho et al. (2014)
Memory Type	Hidden state only	Hidden state + Cell state	Hidden state only
Vanishing Gradient	❌ High risk	✅ Solves with gates and cell state	✅ Solves with fewer gates
Gates Used	❌ None	✅ Input, Forget, Output	✅ Update, Reset
Speed / Training Time	✅ Fast (lightweight)	❌ Slower (more parameters, complex operations)	✅ Faster than LSTM (fewer parameters)
Memory Power	❌ Weak for long-term dependencies	✅ Strong (retains long sequences effectively)	✅ Strong (slightly less than LSTM, but efficient)
Complexity	🟢 Simple	🔴 High (3 gates + cell state)	🟡 Moderate (2 gates, simpler than LSTM)
Performance	❌ Low on long text/tasks	✅ Best for long sequences	✅ Competitive; often as good as LSTM
Best Use Cases	Short, simple sequences	Complex tasks, long text (e.g., translation, Q&A)	Real-time apps, short/medium sequences

 **Table 2: Advanced Comparison – Stacked, Bidirectional, and Variants**

This table compares extended architectures like stacked and bidirectional versions of RNN, LSTM, and GRU.

Feature / Model	 RNN	 Stacked RNN	 Bi-RNN	 LSTM	 Stacked LSTM	 Bi-LSTM	 GRU	 Stacked GRU	 Bi-GRU
 Sequence Flow									
 Memory Type	Short	Slightly better	Short (both dir)	Long (cell state)	Hierarchical long	Full-seq long	Moderate-Long	Hierarchical	Full-seq mod-long
 Trainable Params	🔹 Low	🔹 Medium	🔴 2× RNN	🟡 Medium	🔴 High	🔴 2× LSTM	🟡 Medium	🔴 High	🔴 2× GRU
 Vanishing Gradient	🔴 High	🔴 Severe	🔴 High	🟢 Handled	🟢 Handled	🟢 Handled	🟢 Better than RNN	🟢 Better	🟢 Better
 Exploding Gradient	🔴 Likely	🔴 Severe	🔴 Likely	🟡 Possible	🟡 Controlled	🟡 Controlled	🟡 Moderate	🟡 Moderate	🟡 Moderate
 Architecture Complexity	🔹 Simple	🔹 Deeper	🔹 Bi-path	🟡 Complex	🔴 Deep & gated	🔴 Deep & Bi	🟡 Moderate	🔴 Moderate-deep	🔴 Bi
 Training Time	🟢 Fast	🟡 Slow	🔴 Slower	🟡 Moderate	🔴 Slow	🔴 Slowest	🟢 Faster than LSTM	🟡 Moderate	🔴 Slower
 Feature Learning	🔹 Low	🔹 Moderate	🟡 Bi-context	🟡 Rich	🟡 Deep features	🟢 Richest	🟡 Rich	🟡 Deep Context	🟢 Richest
 Effective Words	🔴 10–20	🟡 20–40	🟢 20–40	🟢 100–200	🟢 200–400	🟢 200–500+	🟢 80–150	🟢 150–300	🟢 150–400
 Real-Time Suitability	✅ Yes	✅ Yes	❌ No	✅ Yes	✅ Yes	❌ No	✅ Yes	✅ Yes	❌ No
 Accuracy Potential	🔴 Low	🟡 Medium	🟡 High	🟢 High	🟢 Very High	🟢 Highest	🟢 High	🟢 Very High	🟢 Highest
 return_sequences Needed?	❌ No	✅ Yes	✅ Yes	❌ No	✅ Yes	✅ Yes	❌ No	✅ Yes	✅ Yes

Implementation of RNN, LSTM and GRU

1. Simple RNN without Pretrained Word2Vec

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

# Load CSV file
df = pd.read_csv("fake_and_real_news.csv")

# Use only first 2000 rows
df = df.head(2000)

# Encode Labels: Fake -> 0, Real -> 1
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# Tokenize text
vocab_size = 5000
maxlen = 100
tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# Train-test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# Define SimpleRNN model
model = Sequential()
model.add(Embedding(input_dim=vocab_size, output_dim=64, input_length=maxlen))
model.add(SimpleRNN(64))
model.add(Dense(1, activation='sigmoid'))

# Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=2)

# Evaluate on validation data
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")
```

2. RNN with Pretrained Word2Vec

```
import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

# 1. Load the dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization
vocab_size = 10000
maxlen = 100
tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Update this path
```

```

w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)
embedding_dim = 300

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Split into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Define the model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False)) # Set True if fine-tuning
model.add(SimpleRNN(64))
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=2)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

3. Stacked RNN

```

import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Make sure it's extracted and accessible
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Stacked RNN Model
model = Sequential()

```

```

model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False)) # Set to True to fine-tune embeddings
model.add(SimpleRNN(64, return_sequences=True)) # First RNN Layer
model.add(SimpleRNN(32)) # Second RNN Layer
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=2)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

4. Bidirectional SimpleRNN with Word2Vec

```

import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Bidirectional

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenize and pad
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Path to the extracted binary file
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Bidirectional RNN model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False)) # Keep embeddings frozen
model.add(Bidirectional(SimpleRNN(64))) # Bidirectional SimpleRNN
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

5. Bidirectional + Stacked SimpleRNN with Word2Vec

```
import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense, Bidirectional

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Ensure this file exists
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Bidirectional + Stacked RNN Model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False)) # Set to True if you want fine-tuning
model.add(Bidirectional(SimpleRNN(64, return_sequences=True))) # First RNN Layer
model.add(Bidirectional(SimpleRNN(32))) # Second RNN Layer
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")
```

6. Simple LSTM + Word2Vec

```
import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
```

```

df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization and padding
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Ensure the file is available
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Simple LSTM model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False)) # keep Word2Vec frozen
model.add(LSTM(64))
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

7. Stacked LSTM + Word2Vec

```

import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization and padding
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

```



```

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Ensure the file exists
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Stacked LSTM model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False))
model.add(LSTM(64, return_sequences=True)) # First LSTM returns full sequence
model.add(LSTM(32)) # Second LSTM returns final output
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

8. Bidirectional LSTM + Word2Vec

```

import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Bidirectional

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Ensure the binary file exists and is extracted
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/test split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# 7. Build Bidirectional LSTM model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False))
model.add(Bidirectional(LSTM(64))) # Bidirectional wrapper
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

9. GRU + Word2Vec

```

import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization and padding
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load pretrained Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Ensure it's present and extracted
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Split data
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build GRU model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False))
model.add(GRU(64))
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate

```

```
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")
```

10. Stacked GRU + Word2Vec

```
import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode Labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Update path as needed
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Split data
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Stacked GRU model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False))
model.add(GRU(64, return_sequences=True)) # First GRU Layer
model.add(GRU(32)) # Second GRU Layer
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")
```

11. Bidirectional GRU + Word2Vec

```
import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GRU, Dense, Bidirectional
```

```

# 1. Load dataset
df = pd.read_csv("fake_and_real_news.csv")
df = df.head(2000)

# 2. Encode labels
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

# 3. Tokenization and padding
vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')
y = np.array(df['label_encoded'])

# 4. Load Word2Vec
word2vec_path = 'GoogleNews-vectors-negative300.bin' # Ensure this path is valid
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

# 5. Create embedding matrix
embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 6. Train/validation split
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 7. Build Bidirectional GRU model
model = Sequential()
model.add(Embedding(input_dim=vocab_size,
                    output_dim=embedding_dim,
                    weights=[embedding_matrix],
                    input_length=maxlen,
                    trainable=False))
model.add(Bidirectional(GRU(64)))
model.add(Dense(1, activation='sigmoid'))

# 8. Compile and train
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32, verbose=1)

# 9. Evaluate
loss, accuracy = model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```

12. Keras Tuner with Word2Vec + LSTM

```

import pandas as pd
import numpy as np
from gensim.models import KeyedVectors
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout
from tensorflow.keras.optimizers import Adam
import keras_tuner as kt

# 1. Load and prepare dataset
df = pd.read_csv("fake_and_real_news.csv").head(2000)
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

vocab_size = 10000
embedding_dim = 300
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(df['Text'])
word_index = tokenizer.word_index
sequences = tokenizer.texts_to_sequences(df['Text'])
X = pad_sequences(sequences, maxlen=maxlen, padding='post')

```

```

y = np.array(df['label_encoded'])

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

# 2. Load Word2Vec and build embedding matrix
word2vec_path = 'GoogleNews-vectors-negative300.bin'
w2v_model = KeyedVectors.load_word2vec_format(word2vec_path, binary=True)

embedding_matrix = np.zeros((vocab_size, embedding_dim))
for word, i in word_index.items():
    if i < vocab_size:
        if word in w2v_model:
            embedding_matrix[i] = w2v_model[word]
        else:
            embedding_matrix[i] = np.random.normal(scale=0.6, size=(embedding_dim,))

# 3. Define model builder for Keras Tuner
def build_model(hp):
    model = Sequential()
    model.add(Embedding(input_dim=vocab_size,
                        output_dim=embedding_dim,
                        weights=[embedding_matrix],
                        input_length=maxlen,
                        trainable=hp.Boolean("trainable_embedding")))

    model.add(LSTM(units=hp.Int("lstm_units", min_value=32, max_value=128, step=32),
                    dropout=hp.Float("dropout_rate", 0.1, 0.5, step=0.1),
                    recurrent_dropout=0.1))

    model.add(Dense(1, activation='sigmoid'))

    lr = hp.Choice("learning_rate", [1e-2, 1e-3, 1e-4])
    model.compile(optimizer=Adam(learning_rate=lr),
                  loss='binary_crossentropy',
                  metrics=['accuracy'])

    return model

# 4. Initialize tuner
tuner = kt.RandomSearch(
    build_model,
    objective='val_accuracy',
    max_trials=5,
    executions_per_trial=1,
    overwrite=True,
    directory='tuner_dir',
    project_name='lstm_finetune_word2vec'
)

# 5. Search for best hyperparameters
tuner.search(X_train, y_train, epochs=5, validation_data=(X_val, y_val), batch_size=32)

# 6. Print best hyperparameters
best_hp = tuner.get_best_hyperparameters(1)[0]
print("Best Hyperparameters:")
print(f"LSTM Units: {best_hp.get('lstm_units')}")
print(f"Dropout: {best_hp.get('dropout_rate')}")
print(f"Learning Rate: {best_hp.get('learning_rate')}")
print(f"Trainable Embedding: {best_hp.get('trainable_embedding')}")

# 7. Train best model
best_model = tuner.hypermodel.build(best_hp)
best_model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=5, batch_size=32)

# 8. Evaluate
loss, accuracy = best_model.evaluate(X_val, y_val)
print(f"Validation Accuracy: {accuracy:.4f}")

```
